

Stateful Reasoning Runtimes:

A Reference Architecture for Dispositional Continuity in LLM Systems

DOI: 10.5281/zenodo.21142562 **ORCID:** 0009-0008-8627-6150

© Tionne Smith, Researcher, Antiparty Press | July 02, 2026 | Reference implementation release: v0.2.0

Canonical claim: Stateless model plus memory retrieval is not identity continuity. Identity continuity requires a dispositional governance layer.

Keywords: stateful AI, identity persistence, LLM architecture, cognitive runtime, dispositional continuity, memory systems, agent orchestration

Abstract

Stateless LLM APIs do not preserve identity continuity across calls. Session replay can preserve recent chronological context, and vector memory can retrieve related historical records, but neither mechanism supplies persistent behavioral governance. This technical note presents Version 2 of a reference architecture for stateful reasoning runtimes, centered on a deterministic offline implementation and evaluation. The implementation compares three runtime patterns: session replay, vector memory, and a dispositional runtime that maintains persistent governance state, bounded trait updates, drift alerts, reset resistance, and constraint persistence. The contribution is narrow: the reference artifact demonstrates that explicit dispositional state can enforce cross-session governance behaviors that replay and retrieval baselines do not provide.

1. Problem Statement Modern

LLM APIs are stateless inference interfaces. A model call receives input context and returns output, but the model invocation itself does not persist a stable identity model, behavioral trajectory, or governance state across calls.

Application builders commonly compensate by externalizing state. The two dominant patterns are chronological replay and memory retrieval. Chronological replay passes recent dialogue back into the model. Memory retrieval stores prior records and retrieves related material at a later time. Both patterns are useful. Neither is equivalent to identity governance.

The architectural problem is that context reconstruction and governance are different operations. Context reconstruction answers questions such as "what was said recently?" or "what prior record is semantically related?" Governance answers different questions: whether a current update should be bounded, whether a trajectory has drifted, whether a constraint still applies under pressure, or whether an identity reset request should be rejected.

The v2 reference implementation tests that distinction directly. It gives replay and memory real baseline implementations, then evaluates whether those baselines can pass governance probes without hidden dispositional state. They cannot.

2. State Taxonomy

The reference architecture separates three state layers

State layer	Function	Implemented baseline
Conversational state	Maintains recent chronological session turns.	SessionReplayRuntime
Associative state	Retrieves related historical records through deterministic local search.	VectorMemoryRuntime
Dispositional state	Maintains persistent governance constraints, bounded updates, trajectory signals, and drift alerts.	DispositionalRuntime

Conversational state is ordered and local to a session. It is suitable for replaying recent turns and preserving immediate context.

Associative state is retrieval-oriented. It is suitable for finding related records across sessions when semantic or keyword overlap is sufficient.

Dispositional state is governance-oriented. It persists structured constraints and metrics that are evaluated before and after a stateless model call. It is not a memory store. It governs whether state changes should be accepted, bounded, flagged, or rejected.

3. Reference Implementation

The reference implementation is deterministic and offline. It does not call an external LLM, embedding API, vector database, or network service by default. **Repository release:**
Stateful-reasoning-runtime-reference

Implemented components:

COMPONENT	ROLE
<code>SessionReplayRuntime</code>	Stores chronological session turns and assembles recent replay context.
<code>VectorMemoryRuntime</code>	Stores memory records and retrieves related records through deterministic bag-of-words cosine search.
<code>DispositionalRuntime</code>	Maintains persistent governance state and evaluates continuity-governance probes.
<code>BoundedAdaptiveDisposition</code>	Bounds trait updates relative to baseline and configured trait limits.
<code>TemporalWeightedDisposition</code>	Applies weighted trait updates when observation confidence is sufficient.
<code>CumulativeDriftDetector</code>	Detects slow-burn drift across a history of observations.
<code>CausalDAG</code>	Represents explicit reasoning dependencies and rejects cycles.
<code>RecalibrationScheduler</code>	Exercises baseline recalibration behavior over accumulated observations.
<code>StatelessPromptAssembler</code>	Assembles prompt context around a stateless model call without invoking a model.
<code>LocalEncryptedStateStore</code>	Exercises local save, export, and delete behavior for dispositional state.
<code>InMemoryVectorIndex</code>	Provides deterministic local retrieval without an external vector database.

The session replay baseline does not use vector retrieval, dispositional state, drift alerts, trait bounds, or governance constraints. The vector memory baseline does not use chronological replay as governance and does not read dispositional state, drift alerts, trait bounds, or identity constraints. The dispositional runtime may use conversational and associative context, but its distinguishing mechanism is persistent governance state.

The local privacy surface is intentionally limited. `LocalEncryptedStateStore` uses a deterministic local XOR stream over JSON state so the proof artifact can exercise save, export, and delete behavior. It is not audited cryptography, production security infrastructure, or a privacy certification.

4. Evaluation Design

The evaluation separates baseline capability probes from governance probes.

Capability probes test whether the baselines perform the jobs they are designed for:

PROBE	EXPECTED CAPABILITY
<code>Recent context replay</code>	Session replay should recover expected current-session turns.
<code>Semantic memory retrieval</code>	Vector memory should retrieve expected related historical records.

Governance probes test what replay and memory lack:

PROBE	GOVERNANCE BEHAVIOR TESTED
<code>Cross-session continuity</code>	Persistent state survives across sessions.
<code>Goal drift detection</code>	Cumulative trajectory changes trigger drift alerts.
<code>Contradictory user-state handling</code>	Large conflicting updates are bounded and logged.
<code>Ethical constraint persistence</code>	Constraints remain active under pressure to optimize engagement.
<code>Identity reset resistance</code>	Reset requests are rejected when they conflict with persistent governance constraints.

The benchmark writes machine-readable receipts to `evals/receipts.json`. Receipts expose replay context, memory retrieval scores, state updates, final dispositional state, prompt assembly output, local state-store behavior, drift alerts, and implemented component lists.

Architecture coverage is checked separately by `evals/verify_architecture.py`, which exercises the named implementation classes directly.

Run commands:

```
python evals/run_eval.py
python evals/verify_architecture.py
```

5. Results

Capability probes:	Governance probes:	Overall:
session_replay: 1/1 replay	session_replay: 0/5	session_replay: 1/7
vector_memory: 1/1 retrieval	vector_memory: 0/5	vector_memory: 1/7
dispositional_runtime: 2/2 context capability	dispositional_runtime: 5/5	dispositional_runtime: 7/7

The result is intentionally split. Session replay passes the replay capability probe. Vector memory passes the retrieval capability probe. Those passes matter because they show that the baselines are implemented as real systems, not inert placeholders.

The same baselines fail the five governance probes because they do not maintain persistent dispositional state. Session replay has recent chronological context, but no durable trait model, drift detector, bounded update strategy, or constraint layer. Vector memory retrieves related records, but retrieval scores do not decide whether a behavioral update should be bounded, whether a trajectory is drifting, or whether an identity reset should be rejected.

The dispositional runtime passes the capability probes and governance probes because it combines context access with explicit governance state: persistent traits, constraints, bounded updates, cumulative drift detection, intervention history, and reset resistance.

6. Scope and Limitations

This benchmark makes a narrow architectural claim. It does not prove general intelligence, therapeutic efficacy, clinical validity, or complete identity preservation.

It is not a production security certification. The local encryption/export/delete surface is a reference interface used to exercise state persistence and deletion behavior in receipts. It should not be treated as audited privacy infrastructure.

The vector memory baseline uses deterministic bag-of-words cosine retrieval. That choice is deliberate: it makes retrieval transparent and reproducible without an external embedding API. It is not intended to represent a production embedding system.

The benchmark is deterministic and offline. It does not evaluate model quality, prompt-following reliability, adversarial robustness of an LLM, or user outcomes. It evaluates whether runtime architecture can preserve governance signals outside stateless model calls.

The current scenarios are minimal. They are sufficient to test the distinction between replay, retrieval, and governance, but they are not a comprehensive benchmark for all possible stateful AI systems.

7. Discussion

The implementation demonstrates that governance state is not reducible to memory. A retrieved memory can provide evidence, but it does not by itself decide whether a new state update is valid, whether a trajectory is acceptable, or whether a persistent constraint should override a current request.

The replay baseline fails governance probes for structural reasons. It stores ordered turns within a session and can replay them. It has no cross-session model of trait evolution, no drift threshold, and no intervention history.

The vector memory baseline also fails governance probes for structural reasons. It stores records and ranks them by deterministic overlap. It can recover related prior material, but retrieval is not equivalent to bounded adaptation or constraint enforcement.

The dispositional runtime adds the missing layer. It models state as something that can be updated, bounded, audited, recalibrated, and used to reject unsafe or inconsistent transitions. That is the architectural distinction under test.

Open work remains. Larger scenario sets should test more varied drift patterns, longer histories, noisy observations, adversarial state updates, and different retrieval methods. A production system would also require stronger privacy infrastructure, user-visible state controls,

independent security review, and model-facing prompt evaluation. Those concerns are outside the scope of this reference artifact.

8. Conclusion

This work provides a reference implementation and deterministic evaluation showing that persistent dispositional state can implement governance behaviors not supplied by session replay or vector memory alone.

References

Smith, T. (2025). Stateful Reasoning Runtimes: Architectural Patterns for Identity Persistence Over Stateless LLM

APIs. Zenodo. DOI: 10.5281/zenodo.17755157

Smith, T. (2025). Dignity-First Artificial Intelligence: Privacy, Ethics, and Human Agency in Stateful Systems. Zenodo. DOI: 10.5281/zenodo.17705201

Smith, T. (2025). Living Thesis: Continuous Validation of Stateful AI Architectures. Zenodo. DOI: 10.5281/zenodo.17280692

Reference implementation release:

<https://github.com/electricwolfmarshmallowhypertext/stateful-reasoning-runtime-reference/releases/tag/v0.3.0>